LAPORAN MODUL 1

Pemrograman Dasar Robotika dengan iMCLab:
Kontrol ON/OFF Motor DC

Akbar Khafi (24081010053)

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR
FAKULTAS ILMU KOMPUTER
PROGRAM STUDI INFORMATIKA
2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Sebuah robot hanya berguna jika gerakannya bisa dikendalikan. Pada tahap paling dasar, pengendalian itu berupa perintah hidup dan mati terhadap aktuator, dalam hal ini motor DC. Setelah kontrol hidup/mati dikuasai, kecepatan motor dapat diatur lebih halus menggunakan Pulse Width Modulation (PWM), yaitu teknik mengatur lebar pulsa dalam satu periode sehingga tegangan rata-rata yang diterima motor berubah.

Praktikum ini memakai kit iMCLab yang dikendalikan mikrokontroler ESP32. Mahasiswa menulis dan mengunggah program lewat Arduino IDE, lalu mengirim perintah dari komputer melalui Serial Monitor untuk menyalakan dan mematikan motor.

1.2 Tujuan

1 Memahami cara ESP32 mengendalikan motor DC melalui driver motor.

2 Memahami cara kerja program `on_off_Motor.ino`, mulai dari konfigurasi pin, PWM, hingga pembacaan perintah serial.

3 Mengamati pengaruh kesesuaian baud rate terhadap keberhasilan komunikasi serial.

BAB II
LANDASAN TEORI

2.1 ESP32 sebagai Pengendali

ESP32 adalah mikrokontroler yang berfungsi sebagai otak sistem: menjalankan program, membaca masukan, dan menghasilkan keluaran digital maupun sinyal PWM. Pada iMCLab, ESP32 menerima perintah dari komputer lewat kabel USB (komunikasi serial) lalu menerjemahkannya menjadi sinyal untuk motor.

2.2 Motor DC dan Driver Motor

Motor DC mengubah energi listrik menjadi gerak putar. Pin GPIO ESP32 tidak mampu menyuplai arus sebesar yang dibutuhkan motor, sehingga dipakai driver motor (rangkaian H-bridge) sebagai perantara. Driver memiliki dua pin arah (input 1 dan input 2) yang menentukan arah putaran, serta satu pin enable yang menentukan apakah motor mendapat daya. Pin enable inilah yang diberi sinyal PWM untuk mengatur daya yang sampai ke motor.

2.3 PWM pada ESP32

ESP32 membangkitkan PWM melalui perangkat keras LEDC. Tiga parameter utamanya adalah frekuensi, resolusi, dan duty cycle. Dengan resolusi 8 bit, duty cycle bernilai 0 sampai 255. Nilai 0 berarti sinyal selalu rendah (motor tidak mendapat daya), sedangkan 255 berarti sinyal selalu tinggi (daya penuh).

2.4 Komunikasi Serial dan Baud Rate

Baud rate adalah laju transmisi data serial. Pengirim dan penerima wajib memakai baud rate yang sama. Jika berbeda, bit data dibaca pada waktu yang salah sehingga karakter yang diterima menjadi rusak atau tidak terbaca.

2.5 Kontrol ON/OFF

Kontrol ON/OFF adalah metode kendali paling sederhana: aktuator hanya punya dua keadaan, menyala atau mati. Pada praktikum ini, karakter `1` berarti ON dan karakter `0` berarti OFF.

BAB III
METODOLOGI

3.1 Metode

Praktikum dilakukan secara eksperimen langsung pada kit iMCLab. Program diunggah ke ESP32, perintah dikirim lewat Serial Monitor, kemudian respons motor, LED, dan keluaran serial diamati.

3.2 Alat dan Bahan

1 Kit iMCLab (ESP32, motor DC, dan driver motor)

2 Laptop/komputer

3 Kabel USB

4 Arduino IDE

3.3 Analisis Program `on_off_Motor.ino`

a. Deklarasi pin dan parameter

Variabel	Nilai	Fungsi
`ledPin`	2	LED bawaan board ESP32
`motor1Pin1`	27	Pin arah motor (input 1 driver)
`motor1Pin2`	26	Pin arah motor (input 2 driver)
`enable1Pin`	12	Pin enable driver, menerima sinyal PWM
`freq`	30000	Frekuensi PWM 30 kHz

Variabel	Nilai	Fungsi
`pwmChannel`	0	Kanal LEDC yang dipakai
`resolution`	8	Resolusi PWM 8 bit (nilai 0 sampai 255)
`dutyCycle`	200	Nilai awal, ditimpa di dalam fungsi `MotorOn()` dan `MotorOff()`
`pin_rpm`	13	Pin pembaca sensor putaran (disiapkan, belum dipakai)

b. Fungsi `setup()`

Fungsi ini berjalan sekali saat ESP32 menyala. Isinya: mengatur pin LED, pin motor, dan pin enable sebagai output; membuka komunikasi serial pada 115200 baud; mengonfigurasi kanal PWM dengan `ledcSetup()`; menghubungkan kanal tersebut ke pin enable dengan `ledcAttachPin()`; lalu mencetak teks "Testing DC Motor...". Fungsi interupsi untuk menghitung putaran (`attachInterrupt`) sengaja dinonaktifkan sebagai komentar.

c. Fungsi `MotorOn()`

Mengisi `dutyCycle` dengan 150, menetapkan `motor1Pin1` HIGH dan `motor1Pin2` LOW agar motor berputar satu arah, lalu mengirim PWM bernilai 150 lewat `ledcWrite()`. Nilai 150 dari 255 setara duty cycle sekitar 58,8%, jadi motor berputar pada sekitar 59% daya, bukan daya penuh.

d. Fungsi `MotorOff()`

Mengisi `dutyCycle` dengan 0 lalu mengirimkannya ke pin enable. Pin arah tetap pada kondisi sama seperti saat ON; yang menghentikan motor adalah hilangnya daya karena PWM bernilai 0.

e. Fungsi `loop()`

Program membaca satu byte dari buffer serial dengan `Serial.read()` secara terus-menerus. Jika karakter yang terbaca `'1'`, program mencetak "LED is on !!!", menyalakan LED pada GPIO 2, dan memanggil `MotorOn()`. Jika karakter `'0'`, program mencetak "LED is off !!!", mematikan LED, dan memanggil `MotorOff()`. Karakter lain diabaikan. Perlu dicatat bahwa `Serial.read()` mengembalikan -1 ketika buffer kosong, sehingga pada kebanyakan putaran `loop()` tidak ada kondisi yang terpenuhi.

3.4 Prosedur Percobaan

Skenario 1 (baud rate 115200)

1 Unggah `on_off_Motor.ino` ke ESP32.

2 Buka Serial Monitor dan atur baud rate 115200.

3 Kirim `1`, amati pesan serial, LED, dan motor.

4 Kirim `0`, amati perubahannya.

Skenario 2 (baud rate selain 115200)

1 Gunakan program yang sama tanpa perubahan.

2 Atur Serial Monitor ke baud rate lain, misalnya di bawah dan di atas 115200.

3 Kirim `1` lalu `0`, amati respons motor.

BAB IV

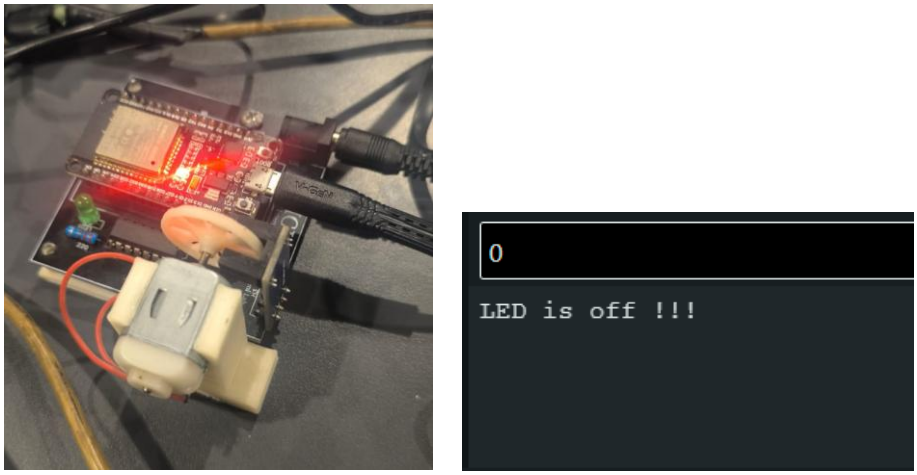
HASIL DAN PEMBAHASAN

4.1 Skenario 1: Pengujian ON/OFF

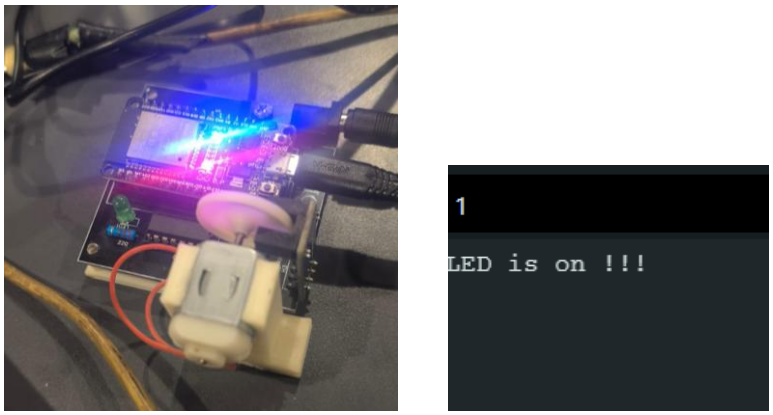
Pada baud rate 115200, perintah `1` memicu pesan "LED is on !!!", LED bawaan board menyala, dan motor berputar. Perintah `0` memicu pesan "LED is off !!!", LED padam, dan motor berhenti.

Perintah	Pesan Serial Monitor	LED board	Motor
1	LED is on !!!	Menyala	Berputar (PWM 150)
0	LED is off !!!	Mati	Berhenti (PWM 0)

Dokumentasi gambar dalam kondisi OFF dan ON:



Gambar 4.1 Kondisi OFF



Gambar 4.2 Kondisi ON

Pembahasan. Teks "LED is on/off" muncul karena program memang mencetak kalimat tersebut. Namun LED di sini bukan sekadar teks: baris `digitalWrite(ledPin, ...)` benar-benar menyalakan LED bawaan pada GPIO 2, sehingga LED berfungsi sebagai indikator visual status perintah, berdampingan dengan motor.

4.2 Skenario 2: Baud Rate Berbeda

Ketika Serial Monitor tidak diatur ke 115200, perintah `1` maupun `0` tidak menghasilkan respons: motor tidak bergerak, LED tidak berubah, dan teks yang tampil di monitor tidak terbaca.

Pembahasan. ESP32 dikunci ke 115200 baud oleh `Serial.begin(115200)`. Bila monitor memakai laju lain, karakter yang sampai ke ESP32 terbaca sebagai nilai yang rusak, sehingga tidak pernah sama dengan `'1'` atau `'0'` dan kedua blok kondisi tidak dijalankan. Jadi penyebabnya adalah ketidakcocokan komunikasi serial, bukan kerusakan motor. Motor tetap berfungsi normal begitu baud rate dikembalikan ke 115200.

BAB V
KESIMPULAN

1 ESP32 mengendalikan motor DC lewat tiga pin: dua pin arah dan satu pin enable yang diberi sinyal PWM.

2 Program `on_off_Motor.ino` menjalankan motor dengan PWM 150 (sekitar 58,8% daya) saat menerima `1` dan menghentikannya dengan PWM 0 saat menerima `0`.

3 Baud rate Serial Monitor harus sama dengan yang diatur di program (115200); jika tidak, perintah tidak dikenali dan motor tidak merespons.

Catatan: fungsi `ledcSetup()` dan `ledcAttachPin()` berlaku untuk ESP32 Arduino core versi 2.x. Pada core 3.x keduanya digantikan oleh `ledcAttach()`.